Medical triage assistant

Team Members:

Student Name	Github
Hector Freard Ruiz	http://github.com/Hectorfr
Ondrej Duba	http://github.com/Ondralol

Date: 23/05/2026

Table of contents

Abstract

In this report, we present a medical triage assistant that determines the most likely medical condition, urgency level, and recommended next steps based on a free-text symptom description. We created a pipeline that utilizes local LLMs to extract the patient's symptoms from their input, combines Retrieval-Augmented-Generation (RAG) and Best Matching 25 (BM25) to retrieve a relevant context from a large medical corpus, and applies a fine-tuned BioBERT classifier to determine the medical urgency based on three levels LOW, MEDIUM and HIGH. All this information is then further processed and presented in a text format to the patient. The primary goal for the assistant is to provide an initial assessment of a patient, while addressing the privacy concerns of cloud-based AI medical systems since everything runs locally. The system is intended as a decision-support tool only and does not constitute professional medical advice.

1. Introduction and Background

These days many people don't have access to a medical care either because it is expensive or because it is inaccessible in their area. Another problem is that when people finally get to hospitals, there are often long queues, and people must often wait for several hours and pay a lot of money to get examined. We propose a solution that can speed up the diagnosis process by introducing a local machine learning pipeline that can triage user's disease and urgency to see a doctor based on their symptoms.

The main idea of the pipeline is that a user will input how they feel, including their symptoms, and our pipeline will output the most likely causes, how urgent their condition is, and suggests the next steps, such as visiting a doctor if the symptoms are serious. This could help many patients identify their condition much faster. This pipeline could be used in two possible scenarios. First, hospitals could use this local pipeline to get patients' approximate diagnosis before getting assigned to a doctor. This would help prioritize patients and shorten waiting times. Second, people in remote areas with a limited access to medical care could run this pipeline locally on any of their devices, such as computers or phones and get medical triage without needing to go to hospital.

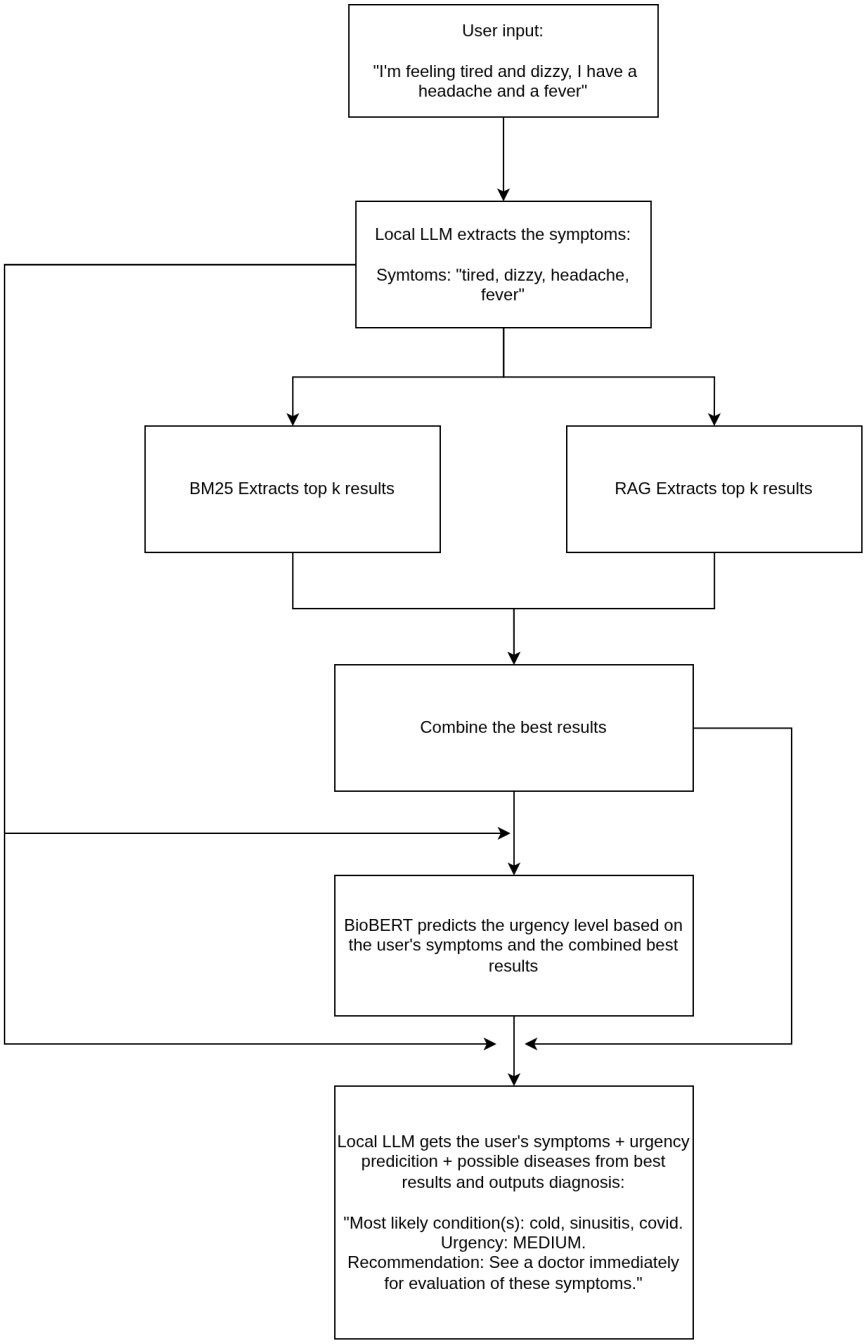
One might say that this approach, to triage a disease based on symptoms, can already be done by modern LLMs, and while this is true, there are a few issues regarding this approach. The first issue is the privacy. When running models such as ChatGPT, Gemini or Claude, your private data leaves your devices and goes through the cloud and for private medical information, this is usually not wanted. Our local medical triage pipeline solves this completely, since the user's private information never leaves the edge device. Another issue that arises from only using LLMs, such as ChatGPT, Gemini or Claude, is explainability. LLMs, even if fine-tuned on medical data, are probabilistic black-boxes that don't reference any concrete sources even if capable of producing reasoning steps. We fix this issue by introducing Retrieval-Augmented-Generation - RAG (Lewis et al., 2020) and Best Matching 25-BM25 (Robertson and Zaragoza, 2009) that contains information about medical diseases and their symptoms directly. We use RAG and BM25 to retrieve relevant medical information based on user's input. This retrieved context is then passed with user's input into Bidirectional Encoder Representations from Transformers (BERT) classifier (Devlin et al., 2019), fine-tuned on medical data, to predict the urgency. The result is a medical triage output that is both explainable and evidence based. All of this runs directly on user's device without any need to connect to the internet.

One thing that we must mention is that this pipeline is intended as a decision support tool and does not replace medical professionals. Users should always consult qualified professionals for clinical decisions.

2 Methodology.

Before explaining the pipeline, let's first explain what RAG and BM25 is. RAG is an AI framework that improves LLMs by giving them access to an external knowledge base. This can provide language models with up-to-date context and reduce hallucinations. BM25 is a keyword search and ranking algorithm used in information retrieval system. The reason we use both RAG and BM25 is that they work on different principles. While RAG requires embedding and uses cosine similarity to search the most relevant results, the BM25 ranks the documents by calculation term frequency and inverse document frequency, essentially providing an exact keyword matching. By combining both approaches, we leverage the strengths of both methods and result in getting a better context.

The pipeline starts with the user's input where the user describes their symptoms in natural language. We then use a local LLM with a custom prompt to extract only the symptoms from the user's input, essentially getting rid of all the unnecessary information. For example, if the user's input is: "I have a headache, a fever and I'm feeling sick and tired", the extracted symptoms will be: "headache, fever, sick, tired". These extracted symptoms are used to extract relevant information from large medical corpus using RAG and BM25. We use the MedQuAD dataset (Ben Abacha and Demner-Fushman, 2019) as the retrieval corpus, which contains disease names in the focus_area field and symptom descriptions in the answer field. For the RAG, we embed the answer field so we can then search for top 2 matching results using the user's symptoms, that we also embed before the search. For the embedder we use medically fine-tuned sentence transformer S-PubMedBert-MS-MARCO (Deka et al., 2022), which was made for data retrieval in the medical field. For the BM25 we don't need to embed the user's symptoms, and we just get top 5 best matching results. We then get the top k best matching result's indices from both RAG and BM25, essentially giving us indices to the original MedQuAD dataset. The k is smaller or equal to 7, because it is possible that both RAG and BM25 return the same indices, so there might be an overlap. We use these indices to extract the records from the MedQuAD dataset. From these records we only take the disease name (from the focus_area field), and then first 175 characters from the answer field. The first 175 characters from the answer field should ideally contain the symptoms description. Because the dataset isn't perfect, it is possible that this is not always the case and it might result in getting results that aren't relevant. After this step we combine the best retrieved results (records filtered to only contain the disease name + 175 characters from the answer field) and append the original symptoms extracted from the user's input via local LLM. This is then fed into BERT Classifier (more on the Training process later), that predicts the urgency level. The last step of the pipeline is combining the original user's symptoms extracted from local LLM, the top retrieved records from RAG and BM25 filtered to only contain the disease name along with the 175 characters from the answer field, and the BERT's classification result. We feed this into local LLM, that creates a structured output, that provides the urgency level, most likely condition and the next steps that the users should do, such as visiting a doctor.



(Medical Triage Pipeline showcase)

For the urgency classifier we used fine-tuned version of BERT-base called biobert-base-cased-v1.2 (Lee et al., 2020). This model was trained on a large biomedical corpus and since it's based on BERT-base, it contains approximately 110 million parameters. For the training, we used two datasets, symptom_to_diagnosis (Gretel AI, n.d.) and Disease and Symptoms dataset (Choong, Q. Z., n.d.). Both datasets contain disease names and their symptoms. After preprocessing steps, which will be explained later, we are left with 1357 records and 41 unique diseases. For each disease, we manually mapped its medical urgency based on the Emergency Severity Index (ESI). The classifier input was constructed by passing each training data point through the same hybrid retrieval pipeline (BM25 and RAG), retrieving relevant context from the MedQuAD dataset, excluding disease name to improve

generalization, and concatenating the original symptoms. We essentially simulated the input of the actual users who will be using the whole pipeline.

The reason why this constructed architecture should outperform baseline LLMs is primarily since we use an external knowledge base through RAG and BM25, which provides explainability and a concrete context. We don't need to purely rely on the LLMs to provide the medical triage but instead use a large medical corpus and essentially compare the patient's medical case to similar medical cases. This approach should also reduce hallucinations, since the final LLM that presents the output to the user has input containing all the symptoms, possible conditions and the urgency, reducing the need to provide unsupported claims.

3 Experiment results and evaluation

The code workflow can be divided into two parts, first, the setup, which contains data preprocessing, BioBERT training and the creation of the RAG and BM25, as well as their corpus, and second, the actual application, that contains the actual pipeline that connects all the components, and user interface.

MedQuAD dataset that was used for BM25 and RAG originally contained 16412 records, but we had to filter it, since some records contained irrelevant information. We only kept the records that contained some information about a disease or its symptoms in the question or the answer field. We also removed unnecessary spacing, URL links and cross reference sections from the text. After this filtering, we were left with 14884 records which were used for the RAG (after embedding). For the BM25 we did some additional processing by first tokenizing the text and then removing all the stop words, which should improve the search accuracy. Even after all the preprocessing, the dataset still contained some noise, but it was minimal.

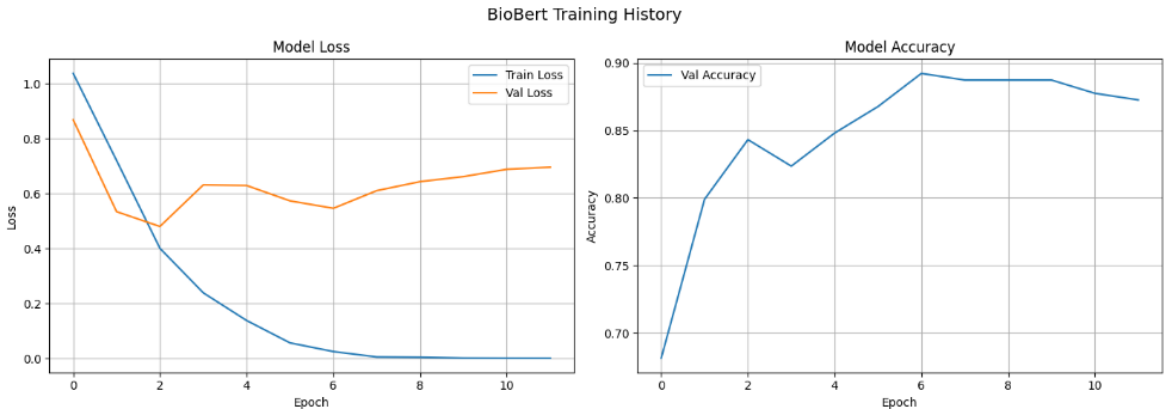
```
res = combined_query_processed("Headache, feeling weak, sore throat, cough, really tired, runny nose")
print(res)
```

```
Disease: cough, symptoms: when you cough, mucus (a slimy substance) may come up.Disease: cold and cough medicines, symptoms: summary : sne
ezing, sore throat, a stuffy nose, coughing -- everyone knows the symptoms of the common cold.Disease: sleep disorders, symptoms: is it ha
rd for you to fall asleep or stay asleep through the night? do you wake up feeling tired or feel very sleepy during the day, even if you h
ave had enough sleep? you might have a sleep disorder.Disease: pleurisy and other pleural disorders, symptoms: pleurisy the main symptom o
f pleurisy is a sharp or stabbing pain in your chest that gets worse when you breathe in deeply or cough or sneeze.Disease: bronchitis, sy
mptoms: acute bronchitis acute bronchitis caused by an infection usually develops after you already have a cold or the flu.Disease: corona
virus infections, symptoms: coronaviruses are common viruses that most people get some time in their life.
```

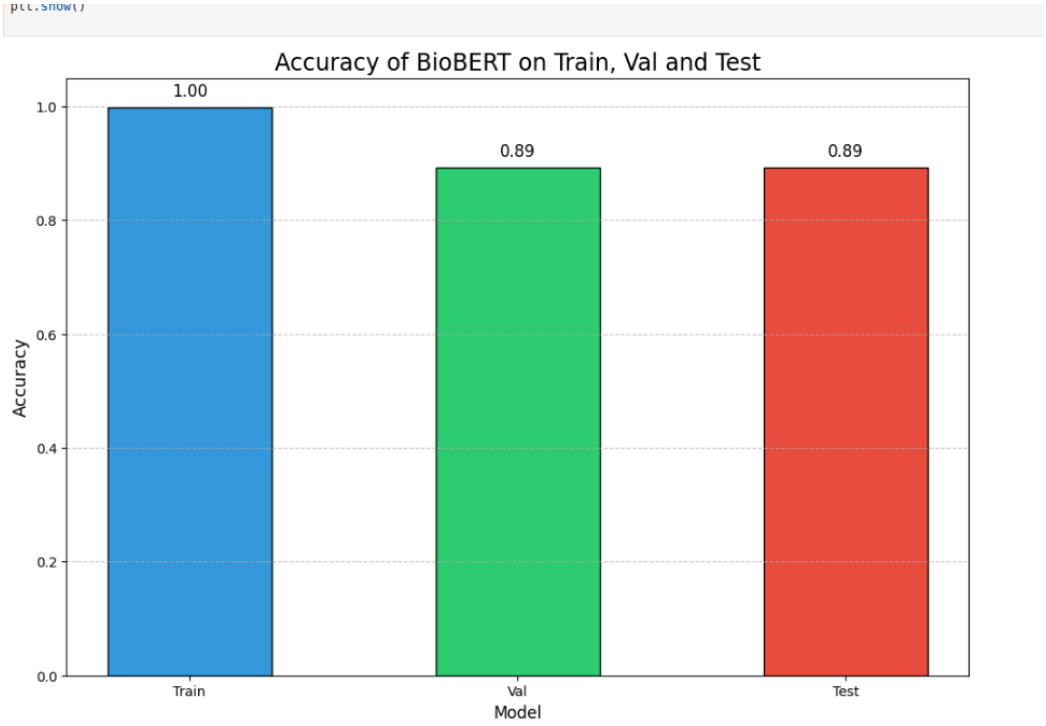
(Showcase of what the combined retrieval context (RAG and BM25) looks like for a specific symptom query)

The BioBERT uses two datasets, that we had to preprocess and concatenate. The symptom_to_diagnosis dataset contains disease name and disease description fields, but the problem is, that the disease description fields are provided in a plain text and we need them in a comma-separated list. We used a local LLM, more specifically llama3.2:3b (Meta 2024) to transform the plain text into comma-separated list. The Disease and Symptoms dataset contained each symptom in a separate field, and we had to combine these into a list. After both datasets had the same structure, we concatenated them, and the last step was dropping the records that contained identical symptoms to prevent data leakage later in the training. We were left with 1357 unique records. We then applied the disease name to urgency level mapping and split the dataset into train (70%), validate (15%) and test (15%) parts.

For the BioBERT training we used the biobert-base-cased-v1.2 with pretrained weights. We set the number of epochs to 15, used the AdamW optimizer with learning rate 2e-5 and weight decay 0.01. We used an early stop mechanism that would stop the training if there was no improvement to the validation accuracy after 5 epochs. We achieved a 1.0 training, 0.89 validation and 0.89 testing accuracies.



(BioBERT Training History)



(BioBERT Accuracy comparison)

Looking at the accuracies, this might seem like a good result, but we have to remember that the dataset only contains 41 unique diseases. By only using the symptoms and not the disease name, we hoped that we could make the classifier generalize so that it could predict an urgency for diseases outside the training dataset. This was partially achieved, but we have no reasonable way to test the actual accuracy and how well it works. We manually found the symptoms of several diseases, that we divided into LOW, MEDIUM and HIGH urgencies ourselves and tested what the classifier would output. The results were accurate in some cases, but wrong in others as you can see in the example.

```python
# Low urgency - basic conditions
print(f"Low urgency: {"-" * 50}")
print(predict_urgency("runny nose, mild cough, sneezing"))
print(predict_urgency("slight itching, dry skin, mild rash"))
print(predict_urgency("mild headache, feeling tired"))

# Medium urgency
print(f"Medium urgency: {"-" * 50}")
print(predict_urgency("yellowing of skin and eyes, dark urine, fatigue"))  # jaundice
print(predict_urgency("wheezing, shortness of breath, chest tightness"))  # asthma attack

# High urgency
print(f"High urgency: {"-" * 50}")
print(predict_urgency("coughing up blood, difficulty breathing, high fever, chills")) # Severe pneumonia
print(predict_urgency("sudden severe chest pain radiating to left arm, sweating, difficulty breathing")) # Heart attack
print(predict_urgency("severe abdominal pain, vomiting blood, rapid heartbeat, dizziness")) # Internal bleeding
print(predict_urgency("sudden weakness on one side of body, slurred speech, drooping face")) # Stroke
```

```
Low urgency: --------------------------------------------------
LOW
MEDIUM
MEDIUM
Medium urgency: -----------------------------------------------
MEDIUM
LOW
High urgency: -------------------------------------------------
HIGH
HIGH
HIGH
LOW
```

(Several test cases of BioBERT Classifier predicting urgencies of several diseases based on the described symptoms)

The Pipeline is created and cached when the application is launched. This initialisation loads up all the necessary components, including the RAG, BM25 and the fine-tuned BioBERT classifier. The pipeline then exposes two key functions used throughout the application: combined_query_processed that retrieves the most relevant context results using the RAG and the BM25, and bert_inference which predicts the urgency level.

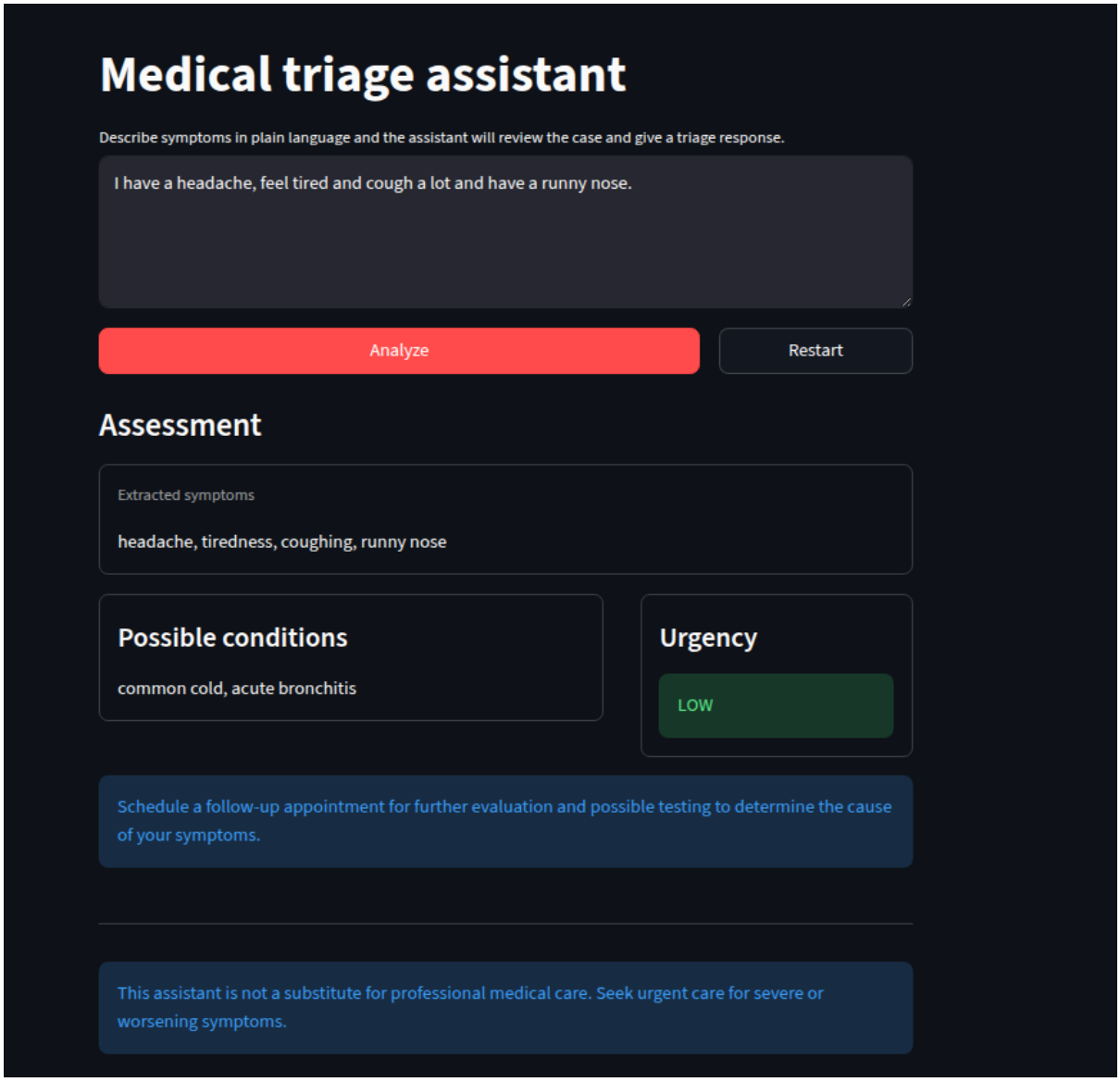
Separately, we created several functions to handle the local LLM queries. There is a function that manages the extraction of the symptoms from the user input and another function that creates the final structured medical triage output from the initial user's input, retrieved context and the BioBERT classifier output. Additionally, we added a validation that checks whether the local LLM model is correctly installed and a function that validates the user's input using LLM, making sure they provided their medical condition.

We had to use prompt engineering across all the LLMs queries to enforce structured outputs and prevent unwanted results. We had to give the LLM precise instructions, otherwise it would deviate from its task. In several cases we wanted to get the outputs in JSON format, and we had to experiment a lot to get the correct results and even then, we still got the wrong results sometimes. We also tried to experiment with preventing user-side prompt injections, which worked most of the times but also created real scenarios where the user input was rejected because the LLM did not think the input was an actual medical condition.

One unsurprising finding that we learned was that the accuracy of all LLM functions depend significantly on the model. Our base model was the llama3.2:3b with 3 billion parameters, but we experimented with smaller and larger models. The smaller models, such as qwen2.5:0.5b (Qwen Team, 2024) that has 0.5 billion parameters performed rather poorly and had some trouble following the specific instructions. On the other hand, larger models, such as gemma4:e2b (Google Deepmind,

2026) with 5 billion parameters performed really well and had no issues following the given instructions. There is an obvious trade-off between the model quality and the inference time. Larger models performed better, but the inference time was noticeably worse.

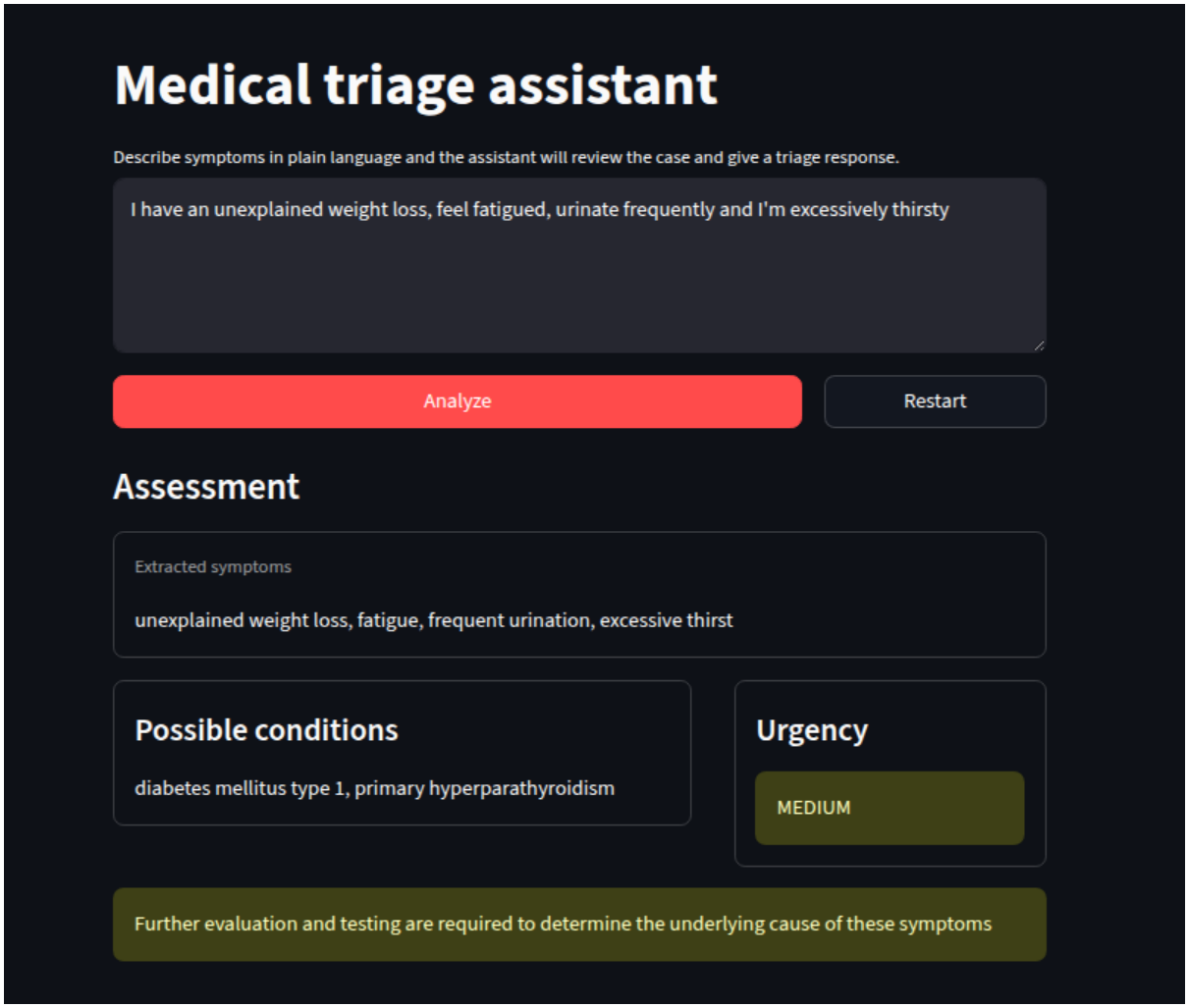
The user interface was built using Streamlit, connecting the main pipeline and LLM helper functions into a single application. It provides the user with clear instructions, input validation warnings, and a structured medical triage output displaying the extracted symptoms, possible conditions, urgency level and recommended next steps.



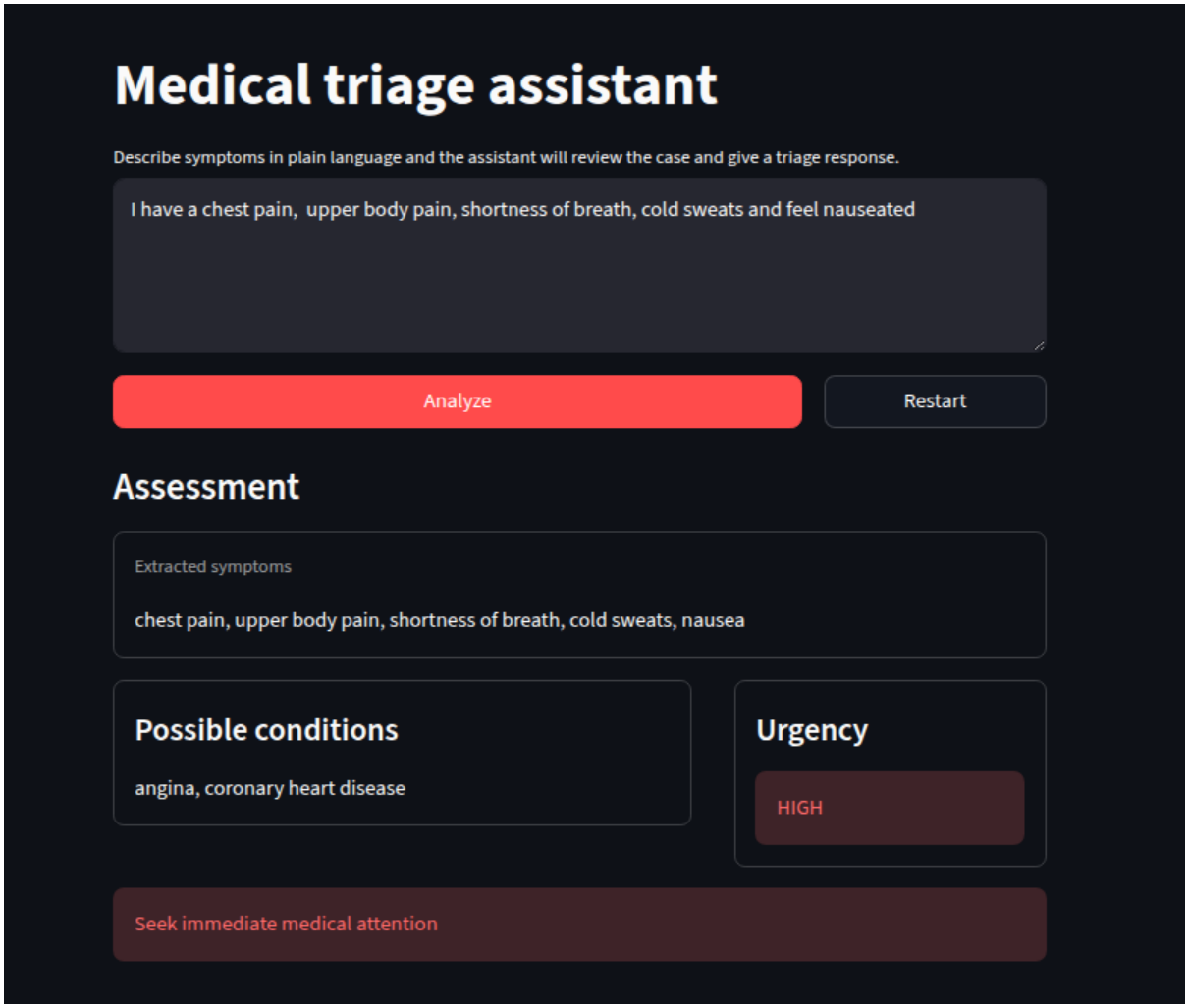
(Example of the UI showing the use of the whole Pipeline and its results)

It is very difficult to measure the accuracy of the whole pipeline because we lack both a suitable evaluation metric and a medical dataset that would contain the exact data that we need, such as medical condition description field, actual disease name, medical urgency and recommended next steps. If we wanted to create such metric, we would have to consider a lot of things, such as what weight should each output have. For example, is medical urgency more important than the recommended next steps? And how would we even compare the recommended next steps in a

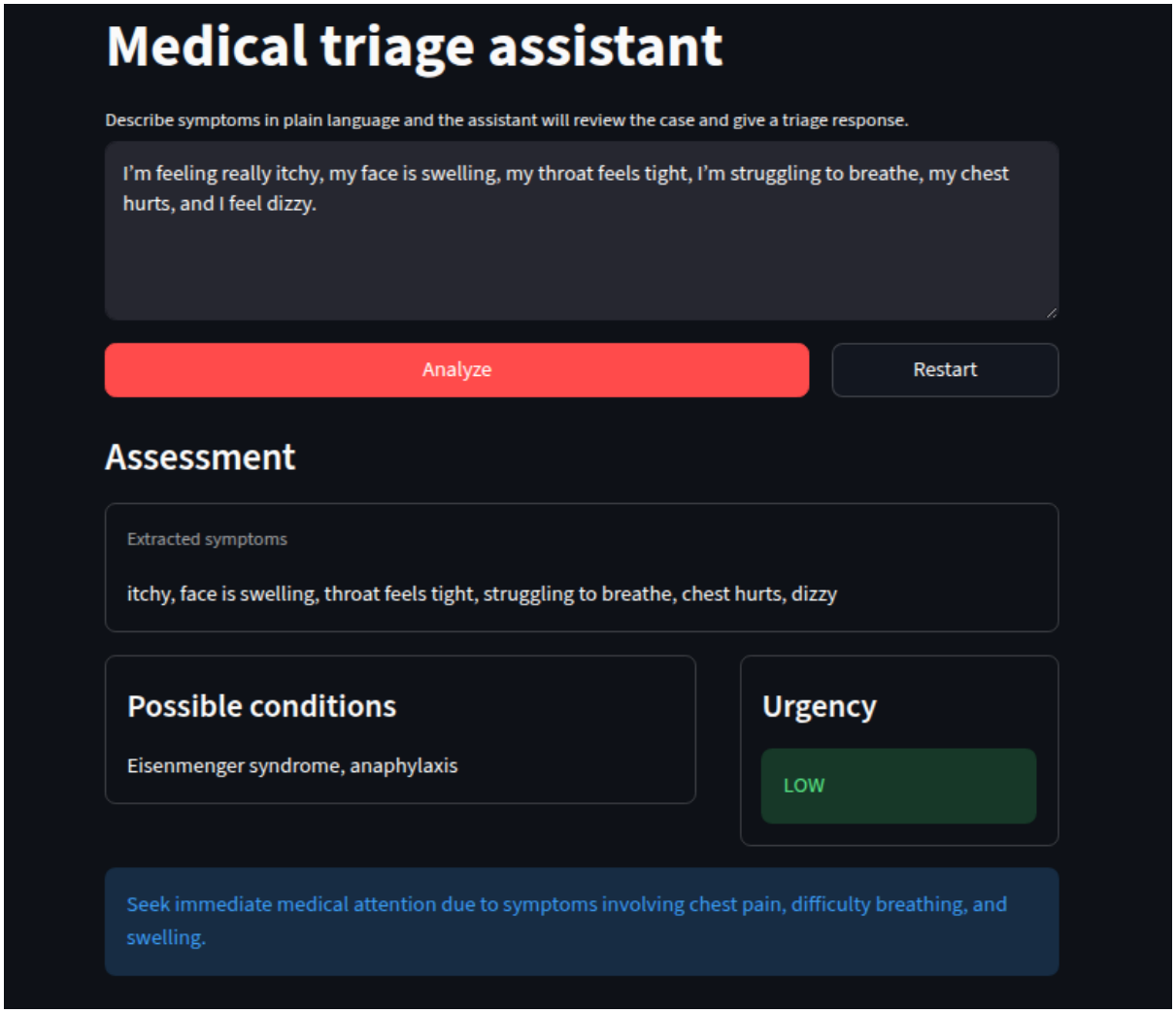
meaningful way. Creating such a metric would take a lot of time and so would take creating a dataset that we could use to test this metric. We can, however, manually search for disease names, their symptoms and map their urgency levels using ESI and compare these results to the pipeline output. Here are a few examples of common medical condition and what our pipeline predicted.



(This example contains the medical symptoms of diabetes, and the model seemed to have correctly predicted the diabetes as one of the conditions, but also suggested hyperparathyroidism, which shares some symptoms, but is not strongly relevant. The MEDIUM urgency is consistent with the ESI)



(This example contains the medical symptoms of a heart attack, and the model seemed to have correctly predicted the condition but also suggested other conditions as well. However, these other conditions are relevant, and the HIGH urgency is consistent with the ESI)



(This example contains the medical symptoms of an allergic reaction, and the model correctly predicted anaphylaxis as one possible condition but also suggested Eisenmenger Syndrome which is completely unrelated. The suggested frequency is LOW, which is incorrect as well, since anaphylactic show might be life threatening)

The repository with all code, experiments and more examples can be found using the following link: https://github.com/Ondralol/nlp_project . The data preprocessing, the creation of RAG, BM25 and BioBERT training can be found inside notebooks/data_processing.ipynb. The source code for the main application is in the /src folder. More information can be found in the README.md file, which also contains a video showcase of the application.

4 Conclusions

To summarise, this project shows a local medical triage assistant built with a combination of several NLP techniques which are combined into a single pipeline. The pipeline utilizes RAG and BM25 to retrieve relevant medical context and uses fine-tuned BioBERT classifier to predict medical urgency. This is then combined with local LLMs that extract the symptoms and piece together the final output to provide the medical assessment. The highlights of this project, which differentiate it from existing solutions, are not only the privacy concerns, which are solved since the whole pipeline can run locally, but also the fact that the provided output is explainable and not based on black-box LLM output. It's clear that the current accuracy of the overall architecture is limited and far from a real use, but with appropriate modifications, it could potentially achieve solid results.

The obvious next steps to improve this pipeline would be to use larger medical datasets, not only for the RAG and BM25, but also for the BioBERT training. If we had more high-quality medical data, the retrieval methods would find more relevant context. For the BioBERT we would not only need a large medical corpus, but also medical professionals that would more precisely label all the diseases and their urgencies. Combining all of this with a larger language model could lead to great results.

5 References

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S. and Kiela, D. (2020) 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks', Available at: https://arxiv.org/abs/2005.11401 (Accessed: 15 May 2026)

- Robertson, S. and Zaragoza, H. (2009) 'The Probabilistic Relevance Framework: BM25 and Beyond', Foundations and Trends in Information Retrieval, 3(4), pp. 333-389. Available at: https://dl.acm.org/doi/10.1561/1500000019 (Accessed: 15 May 2026)

- Devlin, J., Chang, M., Lee, K. and Toutanova, K. (2019) 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding'. Available at: https://arxiv.org/abs/1810.04805 (Accessed: 15 May 2026).

- Ben Abacha, A. and Demner-Fushman, D. (2019). A Question-Entailment Approach to Question Answering. BMC Bioinformatics, 20(1), 511. Available at: https://doi.org/10.1186/s12859-019-3119-4 (Accessed: 15 May 2026)

- Deka, P., Jurek-Loughrey, A. and Deepak, P. (2022) ‘Improved Methods To Aid Unsupervised Evidence-Based Fact Checking For Online Health News’, Journal of Data Intelligence, 3(4), pp. 474–504.

- Lee, J., Yoon, W., Kim, S., Kim, D., Kim, S., Ho So, C, Kang, j. (2020). 'BioBERT: a pre-trained biomedical language representation model for biomedical text mining', Bioinformatics, 36(6), pp. 1234-1240.

- Gretel AI (n.d.) symptom_to_diagnosis dataset. Available at: https://huggingface.co/datasets/gretelai/symptom_to_diagnosis (Accessed: 15 May 2026).

- Choong, Q. Z. (n.d.) Disease and Symptoms Dataset. Available at: https://www.kaggle.com/datasets/choongqianzheng/disease-and-symptoms-dataset (Accessed: 15 May 2026).

- Meta (2024) Llama 3.2: Revolutionizing edge AI and vision with open, customizable models. Available at: https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices (Accessed: 15 May 2026).

- Qwen Team (2024) Qwen2.5: A Party of Foundation Models. Available at: https://qwen.ai/blog?id=qwen2.5 (Accessed: 15 May 2026).

- Google DeepMind (2026) Gemma 4 Model Card. Available at: https://ai.google.dev/gemma/docs/core/model_card_4 (Accessed: 15 May 2026).